# CS 422 Assignment 2: iPerf, TCP Statistics, and Congestion Control

Joseph Getachew, Kushagr Khanna, Isaac Hallman

October 21, 2026

**Repository:** https://github.com/krash3125/cs422-assignment-2. Code links below point at commit `725adf6`.

**Host.** Ubuntu 24.04.5 VM (Multipass on an Apple M1 Pro Mac, 2 vCPU, 4 GB) on home Wi-Fi, host kernel **`6.8.0-147-generic`**. The pipeline runs in Docker (`FROM ubuntu:24.04`) with `--network host --cap-add NET_ADMIN`. Configuration: `tcp_available_congestion_control = reno cubic bbr` (after `sudo modprobe tcp_bbr`), default algorithm `cubic`, qdisc `fq_codel`, `tcp_wmem = 4096 16384 4194304` (Ubuntu default).

**Command:** `./run.sh -n 10 -t 30 -i 0.2 -r 3 --seed 422` (10 destinations, 30 s per test, 200 ms sampling, 3 repetitions, CUBIC/Reno/BBR). Raw data, plots and tables: `results/20261021-082724/`.

## 1. iPerf Throughput Application

### (a) Python iperf3 client

`iperf_client.py` uses only Python's `socket`, `select`, `struct` and `json`; the iperf3 binary is never called. The protocol was taken from the iperf3 source (`iperf_api.h`, `iperf_client_api.c`). Steps in `run_test`:

| Step | Implementation |
|---|---|
| (i) control connection | TCP connect to `host:port`, send a 37-byte random cookie (`make_cookie`). |
| (ii) JSON parameter exchange | On state `PARAM_EXCHANGE (9)`, send the parameters below as 4-byte big-endian length + JSON (`send_json`). |
| (iii) data connection | On `CREATE_STREAMS (10)`, open a second socket, set `TCP_CONGESTION` before `connect()`, send the cookie, and read the algorithm back (`connect`). |
| (iv) transmission | After `TEST_START (1)` / `TEST_RUNNING (2)`, `send_loop` sends 128 KiB blocks for the configured duration. |
| (v) termination | Send `TEST_END (4)`; on `EXCHANGE_RESULTS (13)` exchange results JSON (`recv_json`); on `DISPLAY_RESULTS (14)` send `IPERF_DONE (16)` and close. |

```
{"tcp": true, "omit": 0, "time": 30, "num": 0, "blockcount": 0, "parallel": 1,
 "len": 131072, "pacing_timer": 1000, "client_version": "3.17"}
```

**Robustness.** Every failure becomes a `status` on the returned result instead of an exception, so one bad server never stops the pipeline:

| Situation | Detection | Status |
|---|---|---|
| Non-responsive / overloaded server | no state byte within 15 s (`recv_exact`); `ACCESS_DENIED (-1)` when busy | `CTRL_TIMEOUT`, `ACCESS_DENIED` |
| Server rejects parameters | `SERVER_ERROR (-2)` + `i_errno` (`read_state`) | `PARAM_REJECTED`, `SERVER_ERROR` |
| Connection timeout / refused / DNS | 5 s connect timeout, `ECONNREFUSED`, `gaierror` | `CONNECT_TIMEOUT`, `CONNECT_REFUSED`, `DNS_FAIL` |

| Situation | Detection | Status |
|---|---|---|
| Premature termination | EOF/reset on the control socket or send error on the data socket, checked via `select()` | `PREMATURE_CLOSE`, `SERVER_TERMINATE` |

Partial runs keep their samples. `run_with_retries` retries transient failures with backoff. In all 90 tests the server-reported byte count matched our final `tcpi_bytes_acked` within 0.07%, confirming the protocol works end to end against standard iperf3 servers.

## (b) Destination selection

`servers.csv` is a snapshot of iperf3serverlist.net (183 servers). `select_destinations` shuffles it with a seeded RNG and runs a 3 s probe on each candidate. A failed probe is logged and replaced by the next server, up to `--max-attempts` (40). The shortfall is recorded in `destinations.json`, every attempt in `attempts.csv`.

| # | Destination | IP | Port | Site | min RTT (ms) |
|---|---|---|---|---|---|
| 1 | speedtest.wdc2.us.leaseweb.net | 23.105.9.80 | 5209 | Washington, US | 26.2 |
| 2 | speedtest.dal13.us.leaseweb.net | 209.58.159.68 | 5207 | Dallas, US | 27.0 |
| 3 | 66.35.22.79 | 66.35.22.79 | 30003 | Ashburn, US | 27.2 |
| 4 | 84.17.41.11 | 84.17.41.11 | 5201 | Seattle, US | 52.7 |
| 5 | iperf3.moji.fr | 45.147.210.189 | 5232 | Paris, FR | 96.9 |
| 6 | iperf-ams-nl.eranium.net | 217.18.95.142 | 5202 | Amsterdam, NL | 102.2 |
| 7 | iperf.worldstream.nl | 185.182.195.76 | 5203 | Naaldwijk, NL | 104.7 |
| 8 | 89.187.165.1 | 89.187.165.1 | 5201 | Zürich, CH | 116.7 |
| 9 | speedtest.syd12.au.leaseweb.net | 23.106.244.70 | 5202 | Sydney, AU | 180.7 |
| 10 | 89.187.162.1 | 89.187.162.1 | 5201 | Singapore, SG | 207.7 |

**Outcome: 10 of 10 selected, shortfall 0, 12 probes.** Failures: `122.228.230.122:5205` never sent a state byte (`CTRL_TIMEOUT`, replaced) and `iperf3.moji.fr:5204` was busy (`ACCESS_DENIED`, port 5232 worked). During measurement, 6 attempts failed transiently (3 `ACCESS_DENIED`, 3 `PREMATURE_CLOSE`) and all succeeded on retry, so all 90 tests are valid.

## (c) Goodput

Every 200 ms, `send_loop` reads `TCP_INFO` from the data socket with `getsockopt()` and `make_sample` computes

$$\text{goodput}(t_k) = \frac{\texttt{tcpi_bytes_acked}(t_k) - \texttt{tcpi_bytes_acked}(t_{k-1})}{t_k - t_{k-1}} \cdot 8 \text{ [bit/s]}$$

with $t_k$ from `time.monotonic()` at the actual read. Plots: `q1_goodput_plots`; table: `q1_summary`.

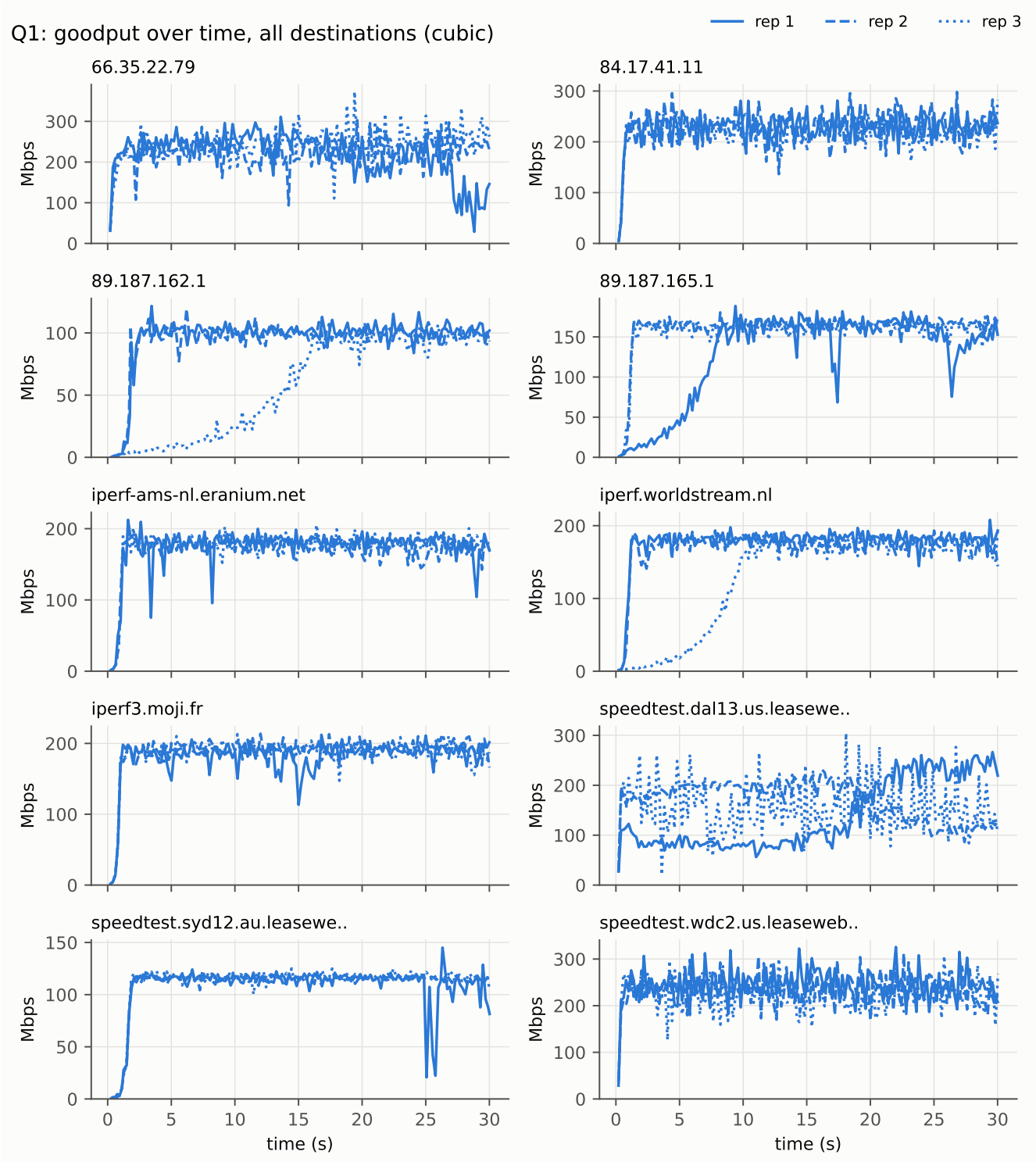



Figure 1: Q1 goodput, CUBIC. One panel per destination, one line per repetition.

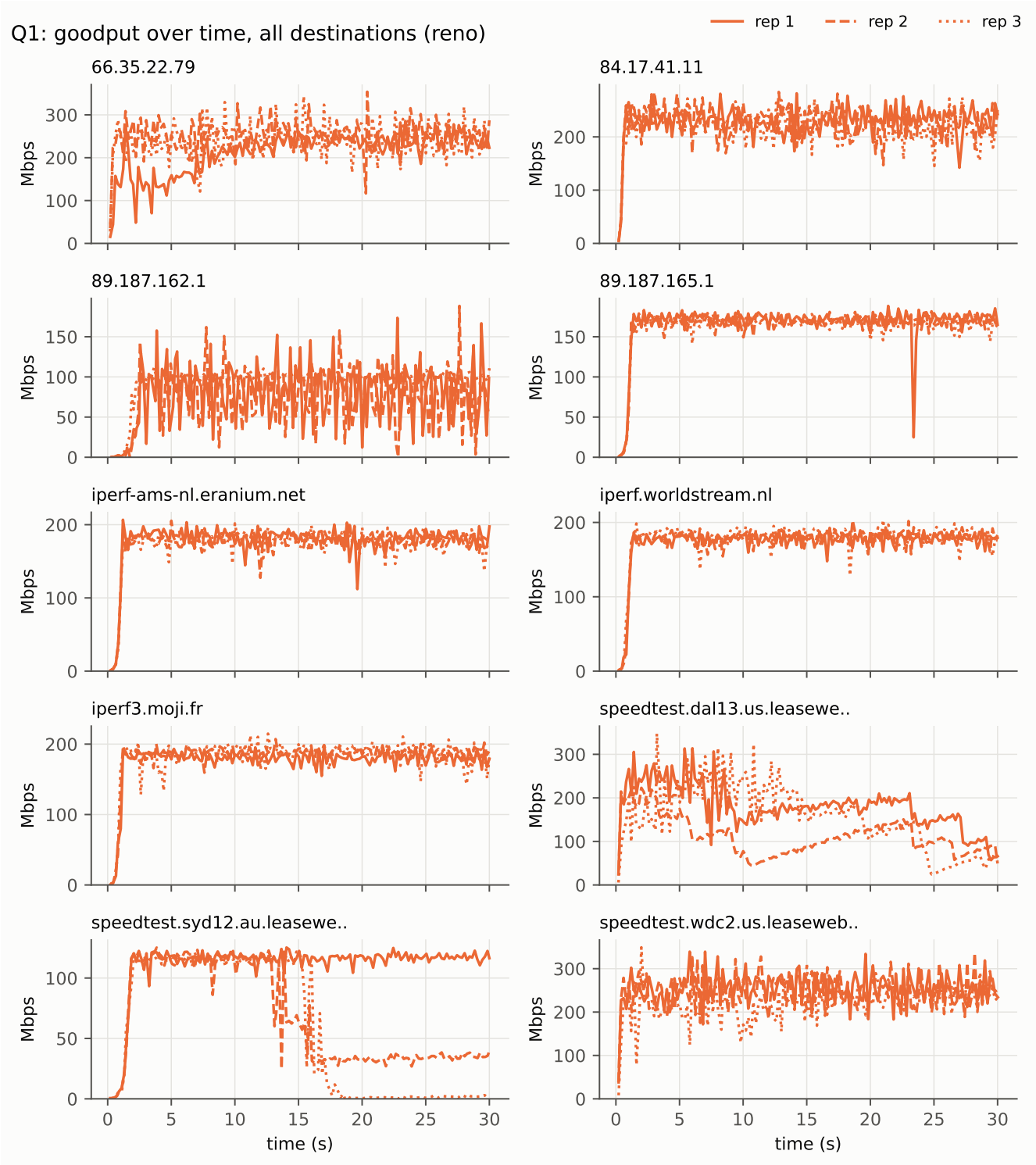



Figure 2: Q1 goodput, Reno.

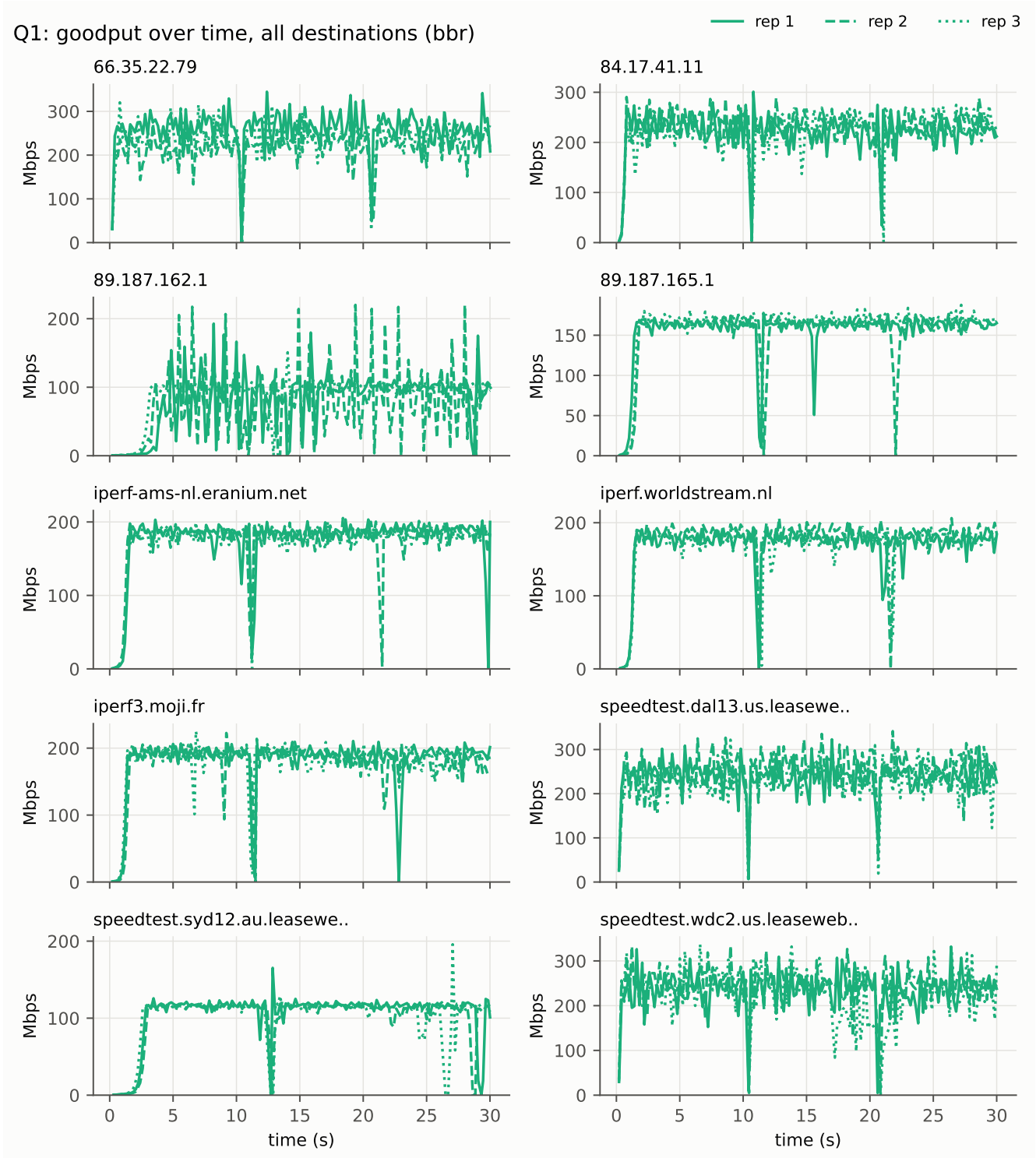



Figure 3: Q1 goodput, BBR.

Goodput over all 200 ms samples of the 3 repetitions (Mbps):

| Destination (min RTT) | CC | min | median | mean | p95 |
|---|---|---|---|---|---|
| Washington (26 ms) | cubic | 28.7 | 233.5 | 233.7 | 289.3 |
| | reno | 8.1 | 242.4 | 241.8 | 302.2 |

| Destination (min RTT) | CC | min | median | mean | p95 |
|---|---|---|---|---|---|
| | bbr | 0.4 | 241.2 | 234.5 | 306.8 |
| Dallas (27 ms) | cubic | 20.3 | 152.8 | 154.4 | 246.8 |
| | reno | 5.7 | 149.2 | 149.5 | 260.4 |
| | bbr | 1.5 | 244.5 | 240.1 | 304.8 |
| Ashburn (27 ms) | cubic | 28.7 | 234.3 | 228.4 | 291.9 |
| | reno | 15.4 | 241.1 | 235.3 | 296.5 |
| | bbr | 1.2 | 245.8 | 239.7 | 303.1 |
| Seattle (53 ms) | cubic | 4.4 | 229.1 | 225.0 | 266.8 |
| | reno | 3.8 | 227.9 | 224.6 | 267.1 |
| | bbr | 0.5 | 228.8 | 223.6 | 269.9 |
| Paris (97 ms) | cubic | 1.1 | 190.1 | 184.7 | 205.5 |
| | reno | 0.5 | 185.9 | 180.4 | 198.9 |
| | bbr | 0.5 | 189.9 | 178.6 | 205.0 |
| Amsterdam (102 ms) | cubic | 1.0 | 178.7 | 172.7 | 193.6 |
| | reno | 0.6 | 181.3 | 175.1 | 194.5 |
| | bbr | 0.5 | 184.5 | 174.0 | 197.8 |
| Naaldwijk (105 ms) | cubic | 0.6 | 176.6 | 158.8 | 190.4 |
| | reno | 1.0 | 180.6 | 174.8 | 192.2 |
| | bbr | 0.3 | 179.9 | 170.2 | 195.4 |
| Zürich (117 ms) | cubic | 0.5 | 162.7 | 148.1 | 173.8 |
| | reno | 0.5 | 169.5 | 164.1 | 179.7 |
| | bbr | 0.3 | 165.5 | 155.8 | 176.3 |
| Sydney (181 ms) | cubic | 0.4 | 115.8 | 110.0 | 121.4 |
| | reno | 0.0 | 113.3 | 78.9 | 121.6 |
| | bbr | 0.0 | 115.0 | 102.4 | 120.6 |
| Singapore (208 ms) | cubic | 0.0 | 97.9 | 82.7 | 107.5 |
| | reno | 0.0 | 97.4 | 84.3 | 124.1 |
| | bbr | 0.0 | 97.1 | 83.1 | 146.4 |

Goodput ranges from about 80 Mbps (Singapore) to about 240 Mbps (US East). The minimum is the first slow-start sample or a stall; 0.0 values occur only on the 180–210 ms paths, where a 200 ms interval can be shorter than one RTT.

# 2. TCP Stats Tracing During Transfer

All Q2 measurements use **CUBIC**, the kernel default. Each test reads the algorithm back from the data socket and every Q2 sample has `cc_used = cubic`.

## (a) Periodic TCP statistics

**Decoding format (kernel 6.8, little-endian).** `tcp_info.py` lists the fields of `struct tcp_info` from `include/uapi/linux/tcp.h` in kernel order, and `decode` unpacks them with `struct.unpack_from` until the returned length (248 bytes here) runs out, so older kernels with a shorter struct are never misread.

| Offset | Type | Field | Logged as |
|---|---|---|---|
| 1 | u8 | `tcpi_ca_state` | `ca_state` (3 = Recovery, 4 = Loss) |
| 68 / 72 | u32 | `tcpi_rtt` / `tcpi_rttvar` (µs) | `srtt_ms`, `rttvar_ms` |
| 80 | u32 | `tcpi_snd_cwnd` (segments) | `cwnd_pkts` |
| 100 | u32 | `tcpi_total_retrans` | `retrans_pkts_delta` |
| 104 | u64 | `tcpi_pacing_rate` (B/s) | `pacing_rate_bps` |
| 120 | u64 | `tcpi_bytes_acked` | `bytes_acked`, goodput |
| 148 | u32 | `tcpi_min_rtt` (µs) | `min_rtt_ms` |

| Offset | Type | Field | Logged as |
|---|---|---|---|
| 160 | u64 | `tcpi_delivery_rate` (B/s) | `delivery_rate_bps` |
| 240 | u16 | `tcpi_total_rto` | `rto_events_delta` |

**Interval measurements.** Counters (bytes acked, retransmits, timeouts) are differenced between consecutive samples and divided by the measured time gap; cwnd, RTT and rates are instantaneous values.

**Loss proxy.** `retrans_pkts_delta`: segments retransmitted in the interval (from `tcpi_total_retrans`). Timeouts per interval and `ca_state` are logged alongside. Retransmits slightly overcount loss (spurious retransmits count too).

**Storage.** `samples.csv`: one row per sample with `run_id`, `rep`, destination, `cc_requested`, `cc_used` and the metrics; units are in column names (`_bps`, `_ms`, `_pkts`) and in `metadata.json`. Writer: `run_all`.

## (b) Visualization

The representative destination is picked automatically (`pick_rep_dest`) as the one with median CUBIC goodput: **Amsterdam** (iperf-ams-nl.eranium.net, min RTT 102 ms). The time series shows its median run; the scatters use all 3 runs. Code: `q2_timeseries`, `q2_scatter`.

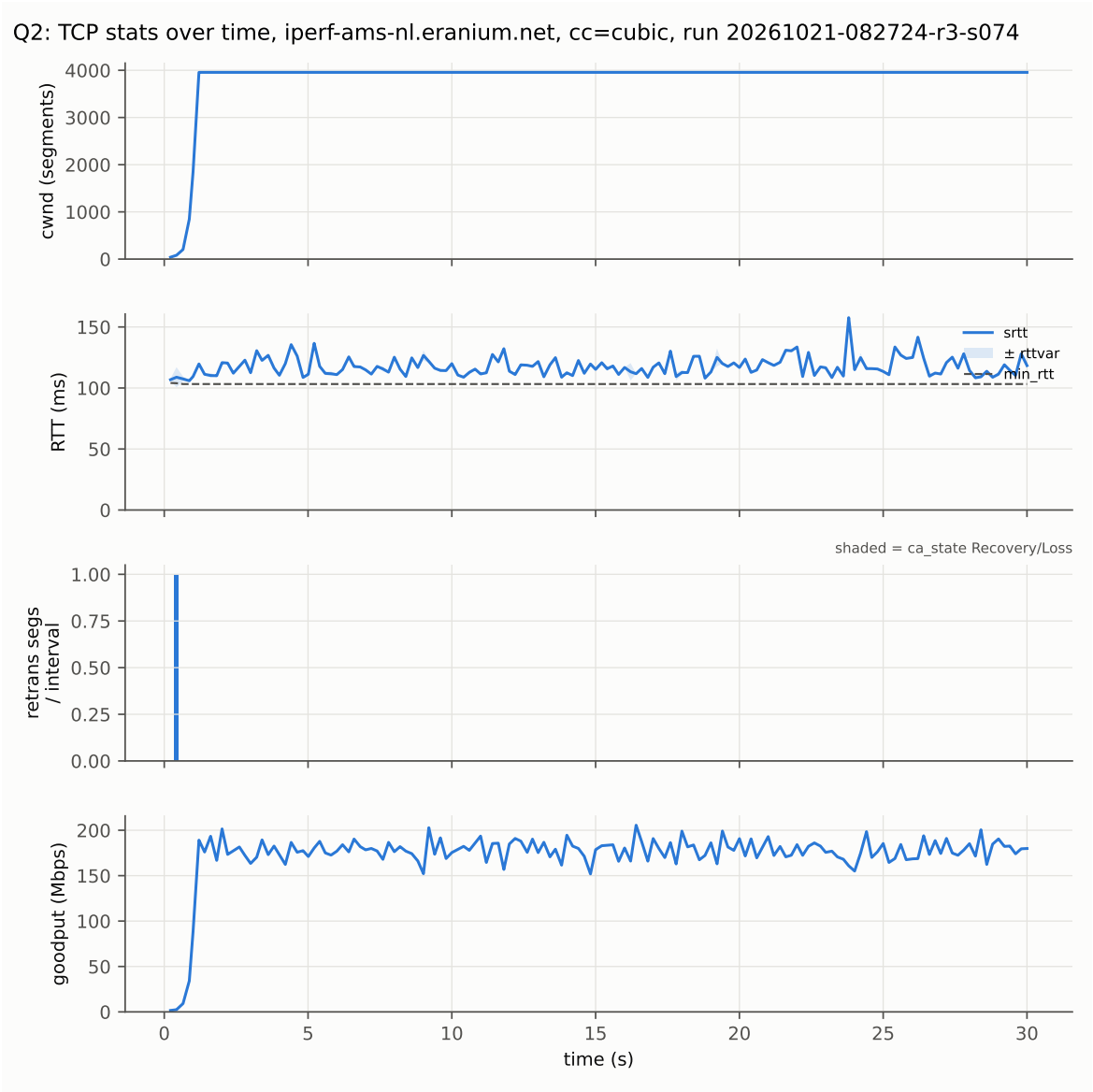



Figure 4: Q2 time series, CUBIC to Amsterdam: cwnd, srtt (dashed = min_rtt), retransmits per 200 ms (shaded = Recovery/Loss), goodput.

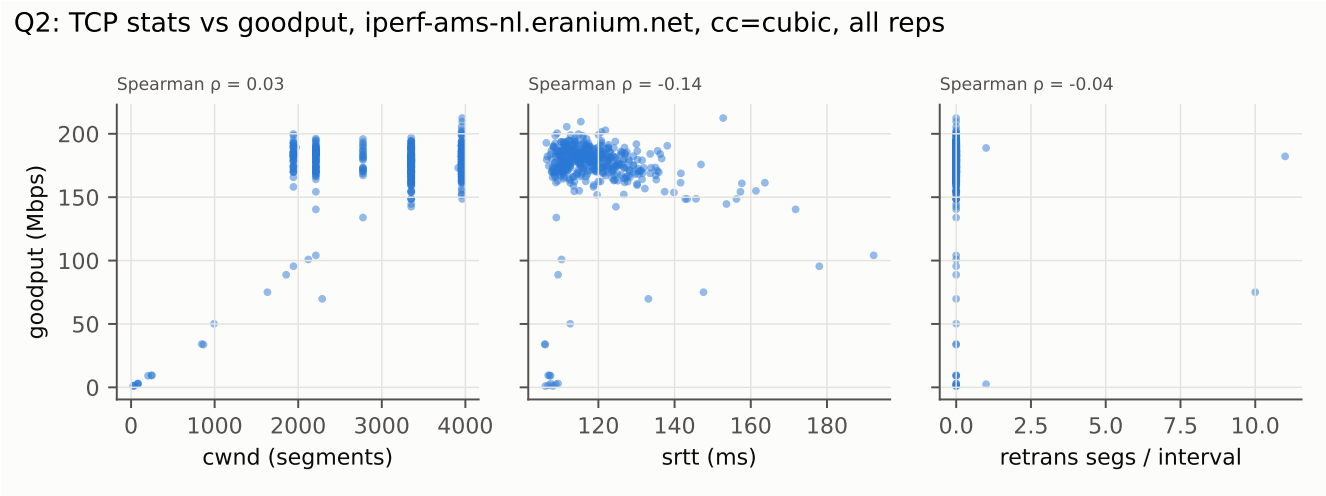



Figure 5: Q2 scatter, all 3 CUBIC runs to Amsterdam, with Spearman ρ.

### (c) Observations

**Metrics and expected effect in congestion avoidance.**

- **cwnd**: segments the sender may have unacknowledged. Goodput ≈ in-flight / srtt, so while cwnd is below the BDP, growing cwnd raises goodput. Above the BDP the link is full and extra window only builds queue.
- **srtt / min_rtt**: smoothed RTT and propagation-delay estimate; srtt − min_rtt is queueing delay. A rising srtt with flat goodput means a growing queue.
- **Retransmits** (loss proxy): non-zero means a buffer overflowed. CUBIC then cuts cwnd to 0.7×, the queue drains and srtt falls, so expected goodput dips briefly after each loss.
- **Goodput**: rate the receiver acknowledges new data.

**What the trace shows.** cwnd grows 10 → ~3950 segments in 1.3 s of slow start (the only region where goodput tracks cwnd), then stays **flat for 29 s** while goodput holds at ~175 Mbps. srtt has a median of 116.6 ms vs 102.7 ms min_rtt (~14 ms queue). Correlations with goodput are near zero: cwnd $\rho = 0.03$, retransmits $\rho = -0.04$, srtt $\rho = -0.14$. In rep 1, CUBIC cut cwnd twice (3962 → 2774 → 1943, ×0.7 each) with no change in goodput.

**Anomalies.**

- *Send-buffer limited, not cwnd-limited.* Median in-flight data is 2.53 MB, far below cwnd, and 72% of samples are flagged `app_limited`: unacked data is capped by the 4 MiB `tcp_wmem` send buffer, not the network, so cwnd stays flat. A rerun with a 32 MiB cap (`results/wmem32m/`) confirmed it — cwnd became the limit, the CUBIC sawtooth and ×0.7 cuts appeared, and cwnd–goodput ρ rose to 0.54.
- *Early slow-start exit (HyStart).* Singapore rep 3 left slow start at 59 segments with no loss and took 20 s to reach the cap: 57.5 Mbps vs 94–96 Mbps in the other reps.

## 3. Comparing CUBIC, Reno and BBR, and a New Design

### (a) Comparison and operating regimes

**Method.** Same 10 destinations, 30 s per test, 200 ms sampling for all algorithms; algorithm set on the data socket before `connect()` and read back (`cc_used = cc_requested` in every test). **3 repetitions**, order: for each rep, for each destination, the three algorithms back to back, rotated each rep (rep 1: cubic → reno → bbr; rep 2: reno → bbr → cubic; rep 3: bbr → cubic → reno). Code: `run_all`, `q3_timeseries_grid`, `q3_scatter_grid`, `q3_dest_summary`.

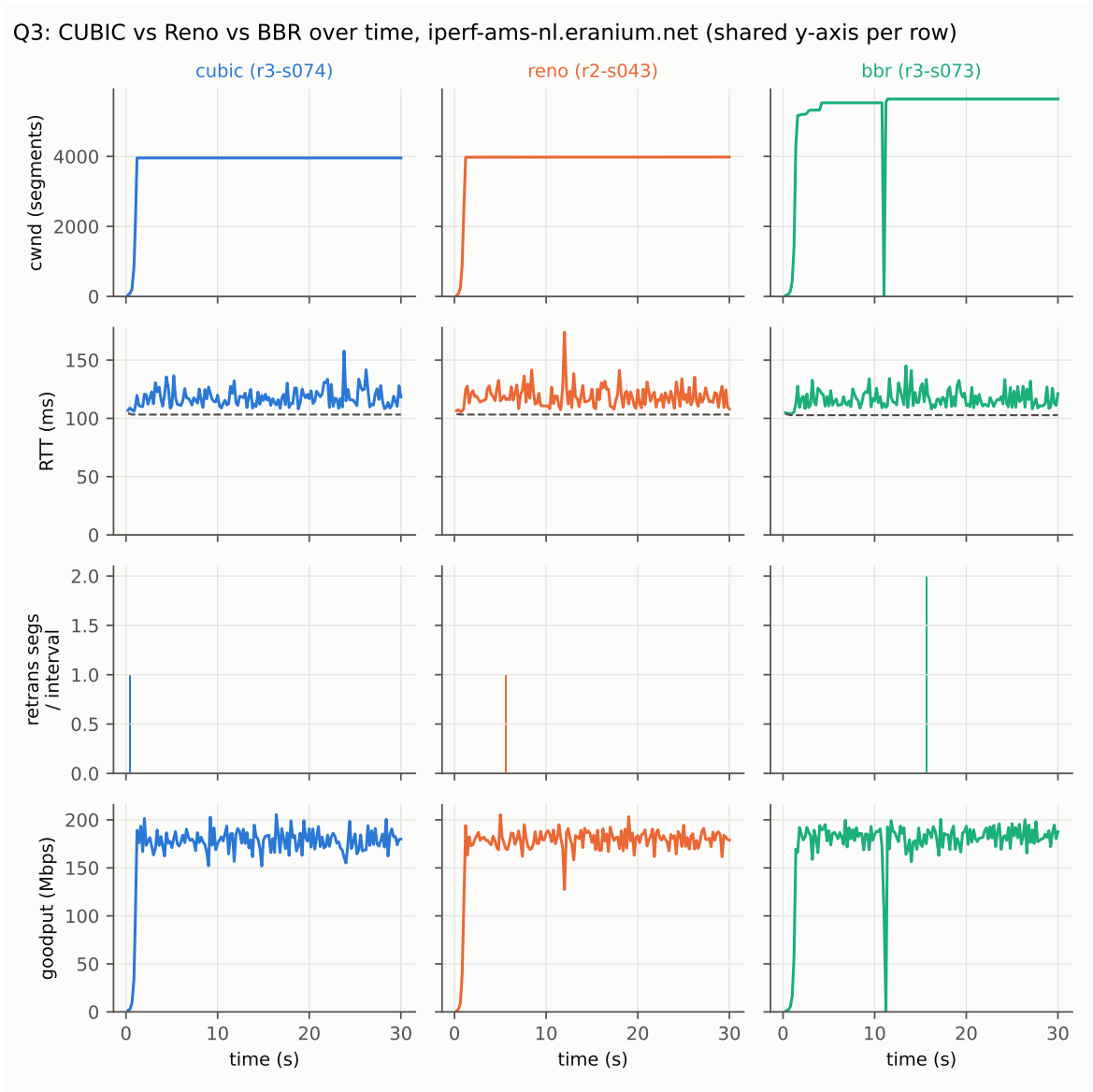



Figure 6: Q3 time series, Amsterdam, median run of each algorithm, shared y-axis per row.

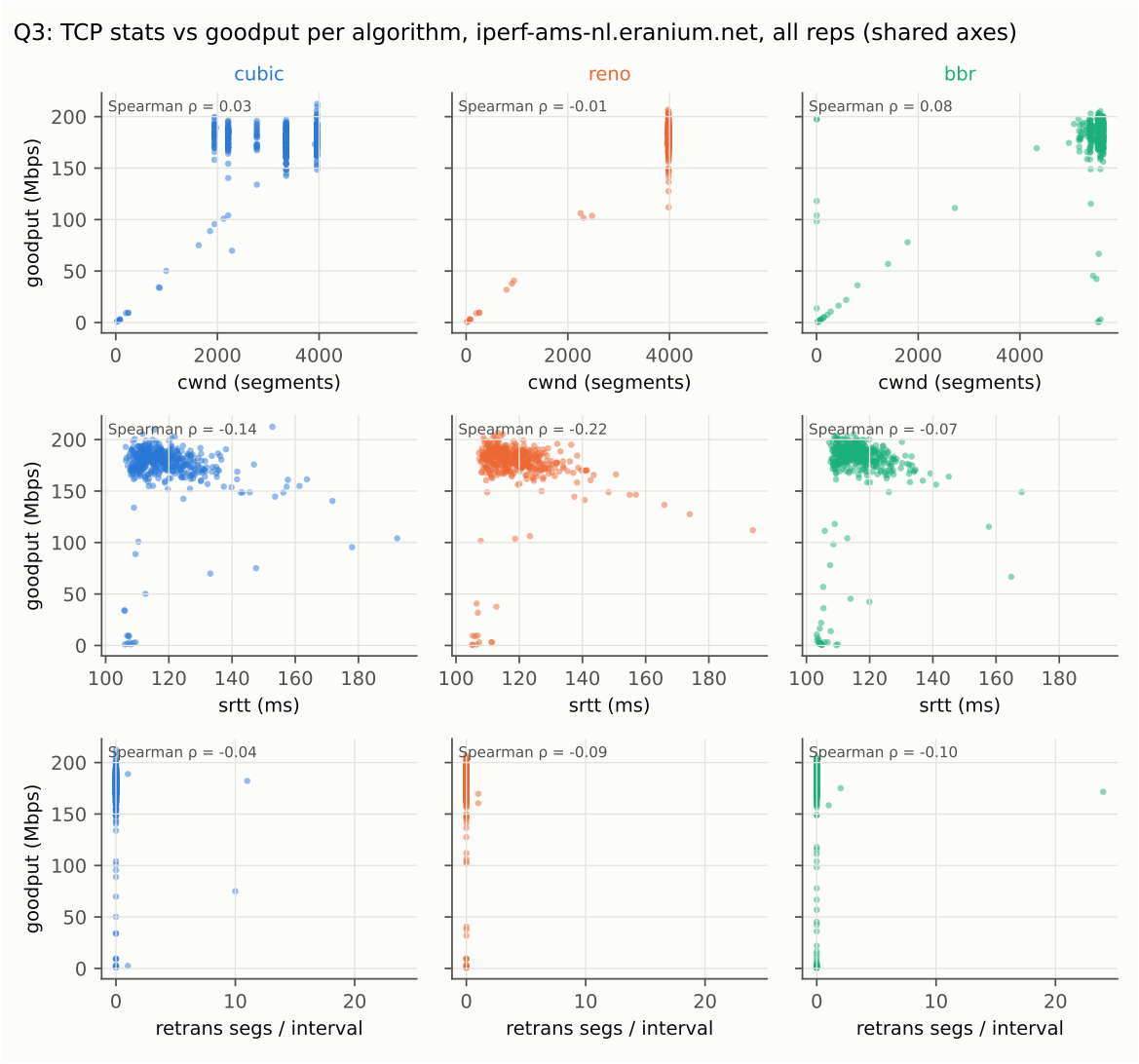



Figure 7: Q3 scatter of cwnd, srtt and retransmits vs goodput, Amsterdam, all reps, shared axes.

Per destination over 3 reps. RTT inflation = median srtt / min_rtt (queueing we add); in-flight = median unacked data:

| Destination (min RTT) | CC | goodput Mbps (mean ± sd) | RTT inflation | retrans % | in-flight MB |
|---|---|---|---|---|---|
| Washington (26 ms) | cubic | 234 ± 10 | 2.90 | 0.07 | 2.3 |
| | reno | 242 ± 16 | 3.01 | 0.06 | 2.3 |
| | bbr | 235 ± 7 | 2.79 | 0.02 | 2.3 |
| Dallas (27 ms) | cubic | 154 ± 15 | 2.09 | 0.09 | 1.0 |
| | reno | 150 ± 34 | 1.32 | 0.20 | 0.6 |
| | bbr | 240 ± 10 | 2.19 | 0.17 | 2.2 |
| Ashburn (27 ms) | cubic | 228 ± 13 | 2.57 | 0.11 | 2.2 |
| | reno | 235 ± 20 | 2.52 | 0.04 | 2.0 |
| | bbr | 240 ± 19 | 2.11 | 0.08 | 1.9 |
| Seattle (53 ms) | cubic | 225 ± 4 | 1.69 | 0.00 | 2.5 |
| | reno | 225 ± 5 | 1.70 | 0.00 | 2.5 |
| | bbr | 224 ± 5 | 1.58 | 0.00 | 2.5 |
| Paris (97 ms) | cubic | 185 ± 4 | 1.12 | 0.00 | 2.5 |

| Destination (min RTT) | CC | goodput Mbps (mean ± sd) | RTT inflation | retrans % | in-flight MB |
|---|---|---|---|---|---|
| | reno | 180 ± 4 | 1.11 | 0.00 | 2.6 |
| | bbr | 179 ± 2 | 1.13 | 0.00 | 2.6 |
| Amsterdam (102 ms) | cubic | 173 ± 2 | 1.14 | 0.00 | 2.5 |
| | reno | 175 ± 2 | 1.13 | 0.00 | 2.6 |
| | bbr | 174 ± 1 | 1.12 | 0.00 | 2.7 |
| Naaldwijk (105 ms) | cubic | 159 ± 26 | 1.09 | 0.00 | 2.6 |
| | reno | 175 ± 2 | 1.10 | 0.00 | 2.6 |
| | bbr | 170 ± 5 | 1.10 | 0.00 | 2.7 |
| Zürich (117 ms) | cubic | 148 ± 17 | 1.06 | 0.00 | 2.6 |
| | reno | 164 ± 2 | 1.07 | 0.00 | 2.7 |
| | bbr | 156 ± 4 | 1.06 | 0.00 | 2.7 |
| Sydney (181 ms) | cubic | 110 ± 2 | 1.03 | 0.02 | 2.7 |
| | reno | 79 ± 29 | 1.03 | 0.12 | 1.3 |
| | bbr | 102 ± 2 | 1.03 | 0.00 | 2.7 |
| Singapore (208 ms) | cubic | 83 ± 22 | 1.02 | 0.00 | 2.6 |
| | reno | 84 ± 9 | 1.18 | 0.04 | 2.7 |
| | bbr | 83 ± 5 | 1.05 | 0.00 | 2.7 |

**Operating regimes.** The host caps in-flight data at ~2.6 MB (Q2c), so whether a path is link- or sender-limited depends on its BDP relative to that cap. Grouping the table above by `rtt_inflation`, `retrans %` and `in-flight`:

1. **Short paths, deep queue** (Washington, Ashburn, Seattle; 26–53 ms). In-flight (2.2–2.5 MB) exceeds the BDP, so the excess sits in the bottleneck buffer: `rtt_inflation` is 1.58–3.01 (bufferbloat). All three saturate the link (~224–242 Mbps, differences within the run-to-run spread). **BBR has the lowest RTT inflation on all three** (2.79/2.11/1.58 vs CUBIC and Reno).
2. **Short lossy path** (Dallas). **BBR 240 Mbps vs CUBIC 154 and Reno 150.** The path shows 0.1–0.2% retransmits; after each loss CUBIC and Reno drop to in-flight 0.6–1.0 MB (Reno's sawtooth is visible in Q1), so the link idles (14–19% of samples cwnd-limited), while BBR keeps 2.2 MB in flight and does not lose throughput to the loss.
3. **Long paths, sender-limited** (Paris → Singapore, 97–208 ms). The ~2.6 MB cap is below the BDP, so the link never fills: inflation 1.02–1.18, almost no loss, and goodput falls as 1/RTT (180 → 83 Mbps). In-flight is 2.5–2.7 MB for every algorithm, so they tie (Amsterdam 173/175/174). The algorithm only matters at startup (CUBIC's HyStart exited early in 5 of 30 runs) and after rare loss: at Sydney a loss burst left Reno at 79 ± 29 Mbps vs CUBIC 110 ± 2.

With a 32 MiB send buffer (4-destination follow-up `results/wmem32m/`, baseline the matched control `results/wmem4m/`), the long paths became network-limited and the algorithms separated: Sydney goodput roughly doubled for CUBIC (95 → 190 Mbps) and BBR (100 → 191), Reno gained less (108 → 153), and BBR retransmitted 6–10% of bytes vs CUBIC under 1%.

**Variability.** BBR is the most repeatable on long paths (sd 1–5 Mbps); its noise is ProbeRTT (cwnd → 4 segments for ~200 ms every ~10 s, the dip at 11 s in the Amsterdam trace). CUBIC's spread comes from HyStart's startup decision, Reno's from whether and when a loss burst happens. On short paths all three vary by 5–20 Mbps as Wi-Fi capacity changes.

**Limitations.** One vantage point (home Wi-Fi is the short-path bottleneck); single flows, so no competition between algorithms; 3 reps of 30 s cannot separate differences below ~10%; public servers are shared with unknown limits, and paths change between runs, so the rotated order and back-to-back runs only reduce, not remove, this bias. "BBR" is Linux BBR v1 (kernel 6.8).

## (b) A hand-written congestion control algorithm: *QAIMD* (queue-aware AIMD with a BDP anchor)

**What the measurements motivate.** (1) CUBIC/Reno fill the buffer on short paths (2–3× RTT) for no extra throughput → target a small, delay-measured queue. (2) Treating random loss as congestion idled the link at Dallas, but ignoring loss entirely made BBR retransmit 6–10% → back off only on loss that comes with a queue or is heavy. (3) Reno's +1 MSS/RTT regrowth

was too slow at Sydney → jump back toward a BDP estimate. (4) Wi-Fi RTT spikes broke HyStart → measure queue from the per-round minimum RTT and require a sustained signal.

**Signals** (all in `TCP_INFO`): `min_rtt` (10 s window), `srtt`, `rtt_round_min` (smallest RTT in the last round), `btl_bw` (max delivery rate over 10 rounds, skipping app-limited samples), `loss` (retransmitted / delivered bytes per round). Derived: BDP = btl_bw × min_rtt, queue $q$ = rtt_round_min − min_rtt.

```
constants: BETA = 0.7, LOSS_TOL = 0.02, MIN_CWND = 4 MSS, PROBE_RTT_EVERY = 10 s

init: state = STARTUP; cwnd = 10 MSS; min_rtt = inf; btl_bw = 0

on every ACK:
    update srtt, rtt_round_min, min_rtt (10 s window)
    update btl_bw (max over 10 rounds, skip app-limited samples)
    pacing_rate = 1.1 * cwnd / srtt

once per round trip:
    bdp   = btl_bw * min_rtt
    q     = rtt_round_min - min_rtt        # standing queue, Wi-Fi spikes removed
    q_thr = clamp(0.15 * min_rtt, 3 ms, 15 ms)
    loss  = retrans_bytes_this_round / delivered_bytes_this_round
    # loss + queue => congestion; loss without a queue => random loss, ignore
    congestive_loss = (loss > 0) and (q > q_thr or loss > LOSS_TOL)

    if state == STARTUP:
        if congestive_loss or (q > 2 * q_thr for 2 rounds)
                           or (btl_bw grew < 25% for 3 rounds):
            state = AVOID
            cwnd  = max(bdp, MIN_CWND)         # drop to pipe size: drains the queue
        else:
            cwnd = 2 * cwnd

    else if state == AVOID:
        if congestive_loss:
            cwnd = max(BETA * cwnd, MIN_CWND)  # multiplicative decrease
        else if q > q_thr:
            # back off in proportion to the excess standing queue
            cwnd = max(cwnd * (1 - 0.5 * (q - q_thr) / srtt), MIN_CWND)
        else if cwnd < bdp:
            cwnd = cwnd + max(1 MSS, (bdp - cwnd) / 4)   # pipe not full: catch up
        else:
            cwnd = cwnd + 1 MSS                # additive increase: probe for more

    if min_rtt not refreshed for PROBE_RTT_EVERY:
        hold cwnd = max(0.5 * bdp, MIN_CWND) for max(200 ms, 1 srtt)   # no full stall

on RTO:
    cwnd = MIN_CWND; state = STARTUP, capped at the last bdp
```

**Design choices.**

- *Throughput:* startup exits to the BDP instead of overshooting, and after a cut the window climbs back to the BDP in a few RTTs (`((bdp - cwnd)/4)`, fixing Reno's slow regrowth (Sydney) and the idle link after cuts (Dallas).
- *RTT:* the delay rule keeps the standing queue near $q_{thr}$ (a few ms), instead of the 2–3× inflation CUBIC/Reno caused. $q_{thr}$ scales with min_rtt; using the per-round minimum and requiring 2 rounds avoids false exits on Wi-Fi spikes.
- *Loss:* only loss with a queue, or above 2%, triggers ×0.7, so random loss is ignored but sustained loss still causes back-off (unlike BBR v1).
- *ProbeRTT* drains to 0.5 × BDP instead of 4 segments, avoiding BBR's 200 ms stall.

**Fairness and convergence.** With the queue below threshold, every flow adds the same step per RTT; every reduction is multiplicative (×0.7 on congestive loss, proportional to excess queue otherwise). Flows sharing a bottleneck see the same queueing delay and reduce together. By the Chiu–Jain argument, additive increase keeps the gap between flows constant and multiplicative decrease shrinks it, so equal-RTT flows converge to an equal share. The operating point is a small limit cycle with total in-flight ≈ BDP + C·$q_{thr}$: the link stays full with only a few ms of queue, independent of router buffer depth. Synchronized ProbeRTT drains let every flow refresh `min_rtt`, so a latecomer cannot lock in an inflated baseline.

**Assumptions.** A single FIFO bottleneck; `min_rtt` observable every 10 s; app-limited delivery samples are flagged (Linux does this); RTT noise smaller than $q_{thr}$; loss without RTT rise is not congestion.

**Limitations.** (1) RTT unfairness: the per-RTT step lets short-RTT flows grow faster, as with Reno. (2) Noisy delay (Wi-Fi jitter, ACK aggregation, reverse-path queueing) can cause false back-offs. (3) A delay-sensitive flow yields to loss-based CUBIC flows that fill the buffer. (4) With shallow buffers loss comes before $q_{thr}$ and it degenerates to AIMD with a BDP anchor. (5) Constants come from single-flow measurements; fairness claims are analytical, not tested.

# Reproducibility

- Inputs: `servers.csv` (server list snapshot); command-line options in `README.md` (`-n` destinations, `-t` duration, `-i` sampling interval, `-r` repetitions, `--seed`).
- Dependencies: the `Dockerfile` (`FROM ubuntu:24.04`) installs Python 3.12, numpy, pandas and matplotlib.
- Host: Linux with `tcp_bbr` loaded (`sudo modprobe tcp_bbr`), measured on kernel `6.8.0-147-generic`. `run.sh` builds the image and runs it with `--network host --cap-add NET_ADMIN`.
- Commands: `./run.sh -n 10 -t 30 -i 0.2 -r 3 --seed 422` runs all three algorithms, saves raw data and generates every PDF plot and table; `./run.sh --analyze-only results/20261021-082724` regenerates plots and tables from saved data.
- Public paths change, so reruns reproduce the method and qualitative findings, not exact numbers.

# Use of AI tools

We used Claude Code as a pair programmer: cross-checking the iperf3 state machine against the iperf3 source, drafting the Python modules and plotting code, setting up the VM and Docker, and drafting report prose. We verified the output by running against real iperf3 servers, comparing server byte counts with `tcpi_bytes_acked`, checking `struct tcp_info` offsets against `linux/tcp.h`, and reading every plot.